CP

HACKATHON SUBMISSION

CivicPulse

AI-Powered Civic Issue Reporting Platform

Problem Statement 2: Community Hero — Hyperlocal Problem Solver

6	14	3,300+	9
AI Agents	API Endpoints	Lines of Code	Google Services

Submitted by	Ronit Sonawane
Affiliation	B.Tech 1st Year, Computer Science & Engineering, IIT Guwahati
Team Size	Solo (Individual Submission)
Repository	`github.com/ronits2407/civicpulse`
Live App	`civicpulse-948264080742.asia-south1.run.app`
Deployment	Google Cloud Run **(asia-south1)**
Date	June 2026

Abstract. CivicPulse is a full-stack, AI-powered civic issue reporting platform designed for Indian cities. Citizens report infrastructure, sanitation, safety, utility, and environmental problems via text, photo, video, or voice. A six-agent LangGraph pipeline powered by Google Gemini 2.5 Flash and Pro automatically classifies, deduplicates, validates, generates resolution briefs, and predicts future civic hotspots. The platform is deployed live on Google Cloud Run with a complete CI/CD pipeline using GitHub Actions and Workload Identity Federation. It supports any typed language via automatic translation, community verification with quorum-based voting, karma gamification, TSP-optimised field team routing, and a fully featured admin dashboard.

Gemini 2.5 Flash/Pro · LangGraph · Next.js 16 · Supabase PostGIS+pgvector · Cloud Run · Mapbox GL

Contents

Project Highlights

- **Solo submission** by a 1st-year B.Tech student at IIT Guwahati
- **6 AI agents** orchestrated by LangGraph with conditional branching and retry/resume
- **3-layer deduplication:** pgvector cosine similarity + PostGIS spatial + Gemini semantic verification
- **Any typed language supported** via automatic detection and translation (Agent 0)
- **TSP-optimised routing** for municipal field teams via Google Maps Routes API
- **Deployed live** on Google Cloud Run with zero-credential CI/CD (Workload Identity Federation)
- **Community verification** with quorum-based voting and karma gamification
- **Predictive hotspot analysis** using geographic DBSCAN clustering + Gemini 2.5 Pro

Evaluation Criteria Quick Reference

Key Insight

Judges: each row maps a scoring criterion to the sections where evidence is presented. This table is intentionally placed **before** the main content for fast navigation.

Criterion	Weight	Key Sections
Problem Solving & Impact	20%	§1 (Problem), §2 (Solution), §8 (User Story)
Agentic Depth	20%	§4 (All 6 agents, conditional edges, retry mechanism)
Innovation & Creativity	20%	§6 (Dual-AI, 3-layer dedup, gamification, TSP routing, tunnel automation)
Google Technologies	15%	§5 (Gemini Flash/Pro/Embedding, Maps Geocoding/Routes, Cloud Run, WIF, OAuth)
Product Experience & Design	10%	§7 (Screenshots, mobile-first design, GitHub-style UI)
Technical Implementation	10%	§9 (AI client, auth flow, community verification, WKB decoding)
Completeness & Usability	5%	§12 (End-to-end flows, tests, health check)

1 Problem Statement and Motivation

1.1 The Challenge

> *"Build a platform that enables citizens to identify, report, validate, track, and resolve community issues through collaboration, data, and intelligent automation. The solution should encourage transparency, accountability, and community participation."*

1.2 Why This Matters

India has over **4,000 urban local bodies** serving 500 million urban citizens. Existing civic grievance redressal systems suffer from systemic failures:

- **Fragmented reporting:** Citizens navigate different portals, phone lines, or offices per issue type. No unified platform exists.
- **No intelligent triage:** A life-threatening open manhole and a faded road marking enter the same queue with identical priority.
- **Duplicate flooding:** The same pothole is reported 50 times. Each becomes a separate ticket, wasting municipal bandwidth.
- **Zero accountability:** Once filed, citizens have no visibility into whether their report was acknowledged or assigned.
- **Language barriers:** India has 22 scheduled languages. English-only platforms exclude the majority of citizens.
- **No predictive capability:** Municipalities are purely reactive — fixing potholes after accidents, clearing drains after flooding.

1.3 How CivicPulse Addresses Every Aspect

Problem	CivicPulse Solution
Fragmented reporting	Single platform: text, photo, video, GPS, and voice — all in one submission
No intelligent triage	6-agent AI pipeline: classifies, deduplicates, validates, generates SLA deadlines
Duplicate flooding	3-layer: pgvector cosine similarity + PostGIS spatial search + AI semantic verification
Zero accountability	Real-time pipeline tracking via Supabase Realtime WebSocket subscriptions
Language barriers	Agent 0 automatically detects and translates typed reports in any language to English
No predictive capability	Agent 5: geographic DBSCAN clustering + Gemini 2.5 Pro for predictive hotspot analysis
Lack of engagement	Karma gamification, leaderboard, community verification, shareable Impact Receipts

2 Solution Overview

CivicPulse is a full-stack AI-powered civic reporting and resolution platform. It accepts reports through multiple modalities, processes them through a six-agent LangGraph pipeline, and provides both citizens and municipal administrators with real-time visibility into every stage of the issue lifecycle.

2.1 High-Level System Architecture

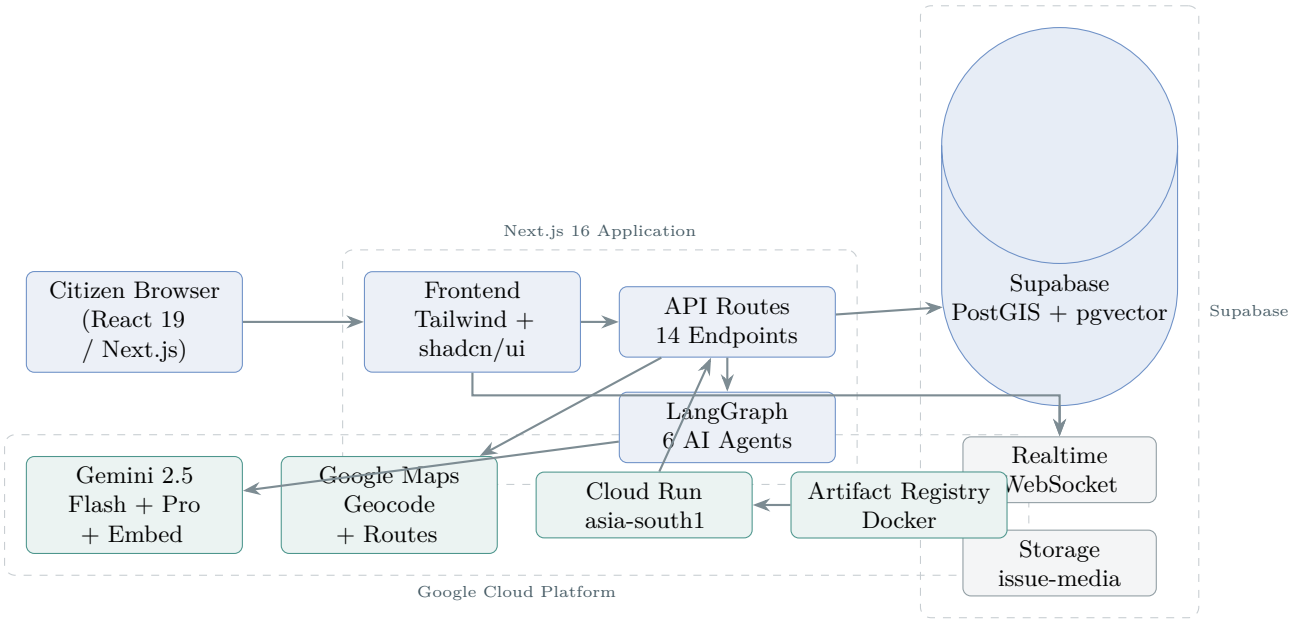

Figure 1: High-level system architecture: citizen browser through Next.js application to GCP and Supabase

2.2 Landing Page

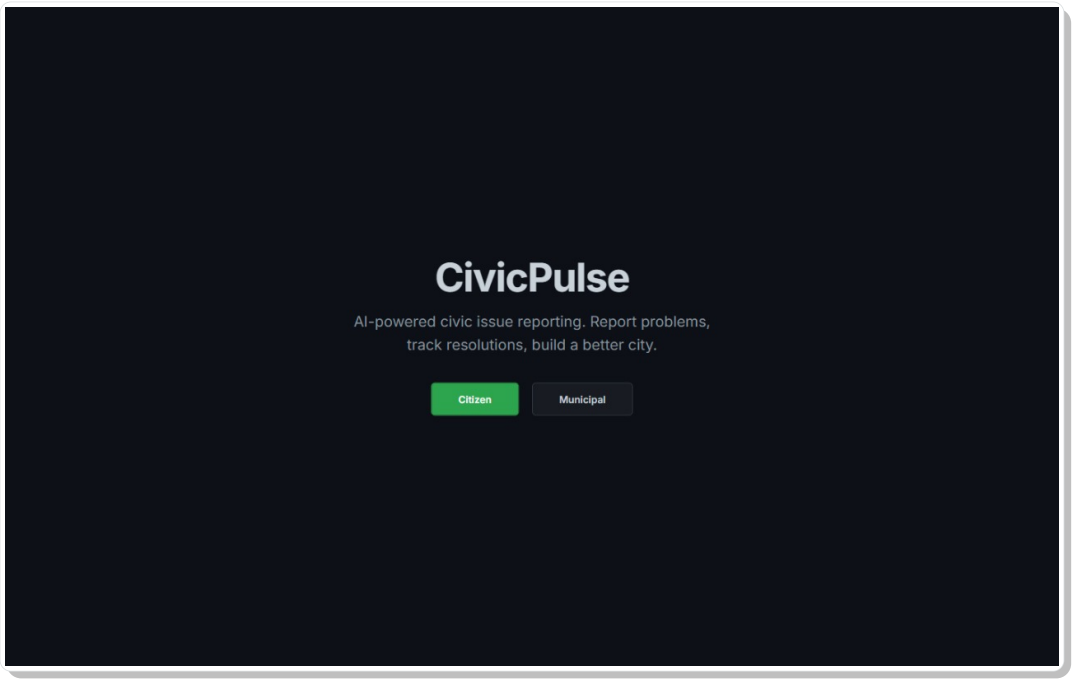

Figure 2: CivicPulse landing page — role selection for citizens and municipal administrators

3 System Architecture

3.1 Technology Stack

Layer	Technology	Version	Purpose
Framework	Next.js App Router	16.2.9	Full-stack React with Turbopack
Language	TypeScript	5.x	Type-safe strict-mode development
Runtime	Bun	Latest	Package management & scripting
UI Library	shadcn/ui (Radix+Mira)	4.11.0	Accessible composable components
Styling	Tailwind CSS	4.x	Utility-first CSS
Animation	Framer Motion	12.41.0	Transitions & micro-interactions
Maps	Mapbox GL JS	3.25.0	Interactive map rendering
AI (Primary)	Google Gemini	2.10.0	Flash/Pro for all 6 agents
AI (Alternate)	OpenAI SDK (Ollama)	6.45.0	Self-hosted model support
Agent Orchestration	LangGraph	1.4.6	StateGraph with conditional edges

Layer	Technology	Version	Purpose
Database	Supabase (PostgreSQL 17)	—	PostGIS + pgvector data store
Auth	Supabase Auth	—	Google OAuth + Email magic link
Geocoding	Google Maps Geocoding	—	Reverse geocoding (lat/lng → address)
Routing	Google Maps Routes API	—	TSP-optimised field team routing
Weather	Open-Meteo API	—	Historical weather data (free, no key)
Clustering	Geographic DBSCAN (custom)	—	Haversine-based district-constrained
Geo Parsing	wkx	0.5.0	PostGIS WKB/WKT geometry decoding
Deployment	Google Cloud Run	—	Serverless container hosting
CI/CD	GitHub Actions	—	Automated build & deploy pipeline
Container	Docker (multi-stage)	—	node:20-alpine production image

4 Agentic AI Pipeline — Deep Dive

The core of CivicPulse is a **six-agent pipeline** orchestrated by LangGraph. Each agent is a discrete node in a `StateGraph` reading from and writing to a shared `AgentState`. The pipeline uses conditional edges to short-circuit at any stage.

4.1 Pipeline Architecture

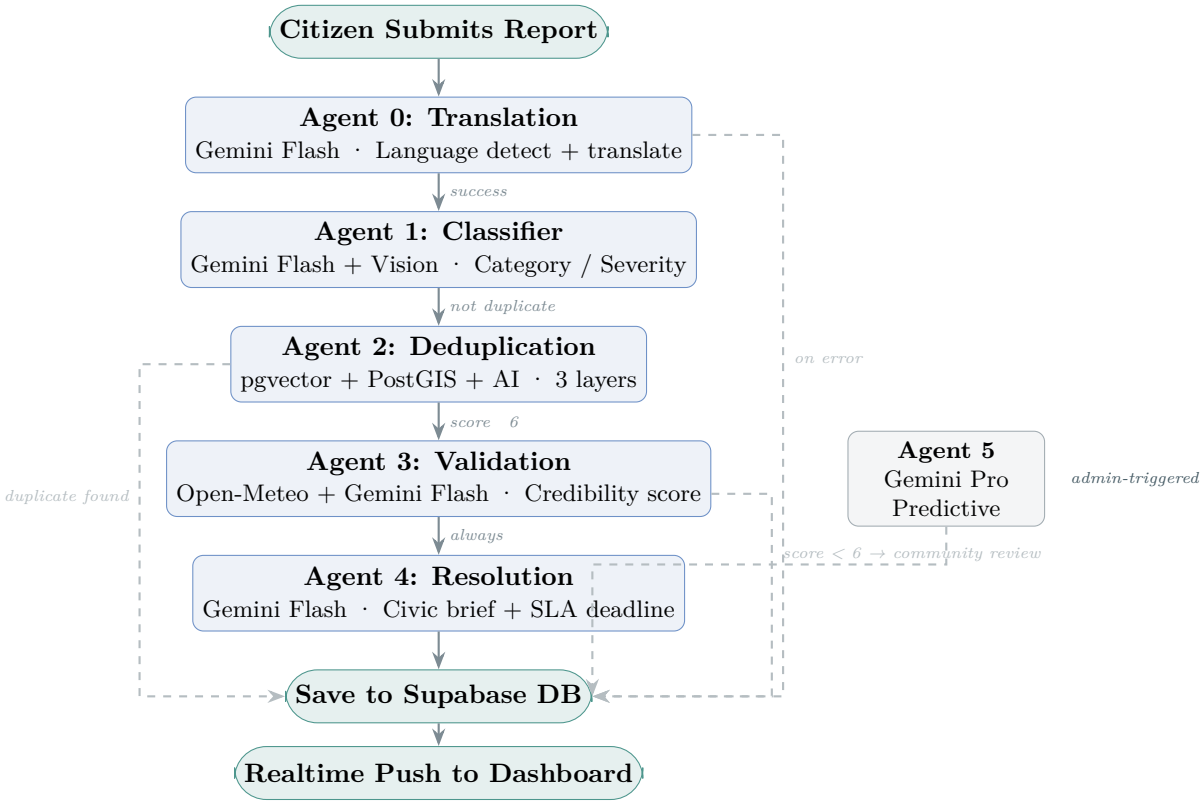

Figure 3: LangGraph StateGraph pipeline with conditional edges and Agent 5 async path

4.2 Pipeline Stage Tracking

Every issue has a `pipeline_stage` column updated in real time. The citizen dashboard subscribes via Supabase Realtime and renders a visual progress stepper.

Stage Value	Meaning
`agent0_translation`	Language detection in progress
`agent1_classifier`	Classification in progress
`agent2_deduplication`	Duplicate check in progress
`agent3_validation`	Credibility scoring in progress
`agent4_resolution`	Civic brief generation in progress
`completed`	Full pipeline completed successfully
`awaiting_community_review`	Credibility < 6; awaiting citizen votes
`review_rejected`	Community voted to reject the report
`*_failed`	Agent failed; eligible for retry/resume

4.3 Agent 0: Translation

File: `lib/agents/agent0-translation.ts`

Detects the language of a citizen's typed report and translates it to English so all downstream agents operate on a common language. Gemini Flash returns structured JSON: `{detected_language, translated_text, thought_process}`. The original language and translation trace are stored for audit purposes. This means a citizen typing in Marathi, Tamil, or Bengali receives identical AI processing quality as one typing in English.

> **Note**
>
> **Voice input note:** The browser's Web Speech API (`webkitSpeechRecognition`) recognises speech in **English** (en-US) only — it does not reliably transcribe Indian regional languages. Citizens who speak Marathi, Hindi, or Tamil should *type* their report; Agent 0 will then detect and translate automatically. A toast notification in the UI informs users if their browser does not support voice input at all.

4.4 Agent 1: Classifier

File: `lib/agents/agent1-classifier.ts`

Multimodal classification: If an image is attached, the agent first downloads it, optimises via `sharp` (1024px, JPEG 85%), and sends it as base64 inline data to Gemini Flash Vision. The visual description is combined with the (translated) text and sent for classification.

Output fields: `category` (one of: infrastructure, sanitation, safety, utility, environment), `subcategory`, `severity` (1–10), `is_emergency` (boolean), `suggested_title`, `department_id` (matched from the `departments` table by category scope).

4.5 Agent 2: Deduplication — Three-Layer Strategy

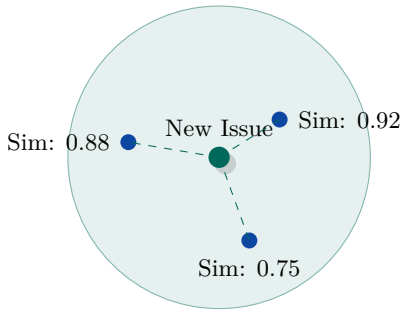

Figure 4: Agent 2 queries PostGIS for issues within 200m, then applies pgvector cosine similarity. Issues ≥ 0.85 are flagged as duplicates.

File: `lib/agents/agent2-deduplication.ts`

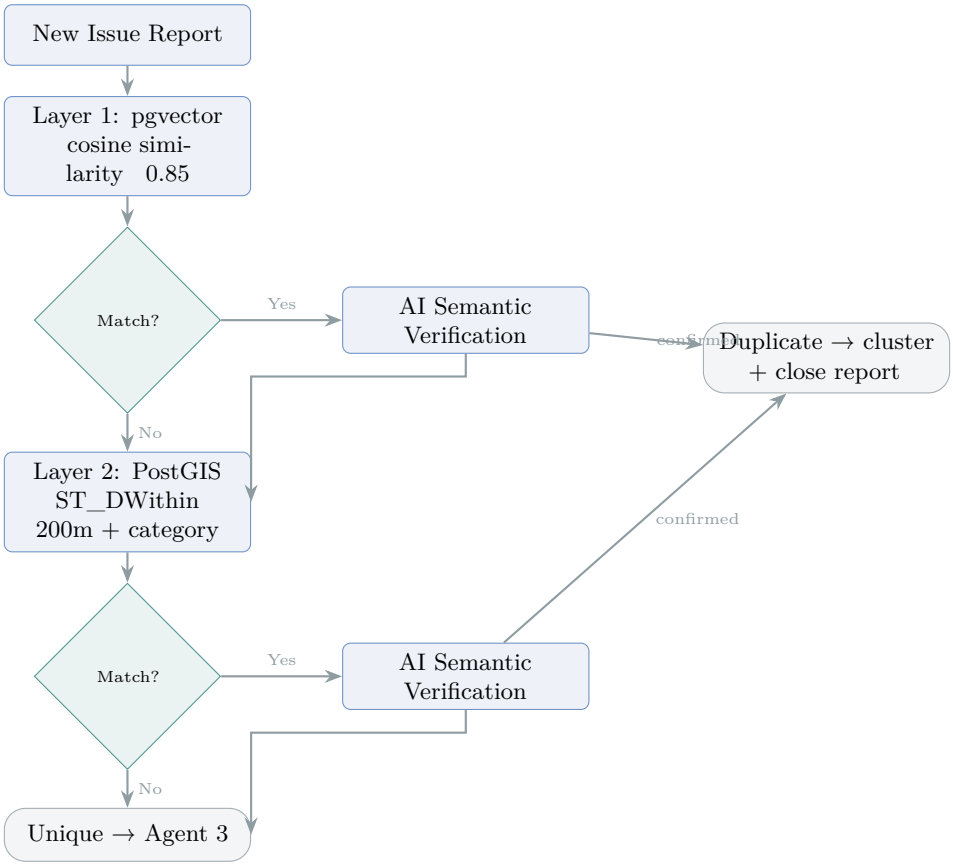

Figure 5: Agent 2 three-layer deduplication: vector similarity → spatial proximity → AI semantic verification

Why three layers: Vector similarity alone produces false positives (similar language, different locations). Spatial proximity alone misses differently worded reports. AI verification is the final semantic judge — "Traffic light broken at MG Road" vs "Pothole near MG Road signal" are correctly identified as *different* issues despite overlapping in both vector space and geography.

4.6 Agent 3: Validation

File: `lib/agents/agent3-validation.ts`

Two-loop reasoning:

1. **Weather relevance check:** Asks Gemini if the issue type needs weather context (waterlogging = yes; broken streetlight = no).
2. **Conditional weather fetch:** If required, fetches live data from Open-Meteo (temperature, precipitation, wind) — free, no API key.
3. **Credibility assessment:** Gemini scores the report 1–10 considering: weather consistency, description verifiability, severity proportionality, and spam signals.
4. **Routing:** Score $\geq 6 \rightarrow$ Agent 4. Score $< 6 \rightarrow$ community review queue with push notification to nearby citizens.

4.7 Agent 4: Resolution

File: `lib/agents/agent4-resolution.ts`

SLA matrix:

Category	High (7–10)	Medium (4–6)	Low (1–3)
Infrastructure	24 h	72 h	168 h
Sanitation	12 h	48 h	96 h
Safety	6 h	24 h	72 h
Utility	12 h	48 h	96 h
Environment	48 h	120 h	240 h

Generates a 150–200 word professional civic action brief. If the original language is non-English, also generates a `local_civic_brief` translated into the citizen's language.

4.8 Agent 5: Predictive Intelligence

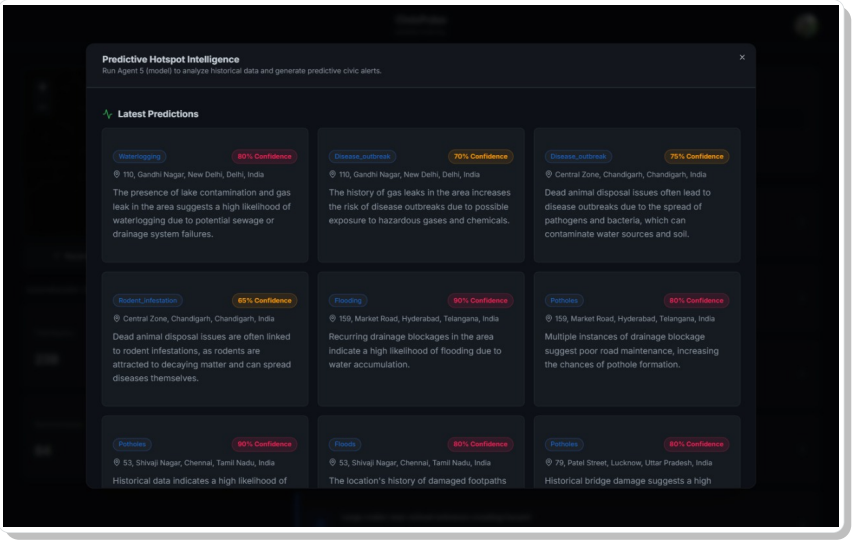

Figure 6: Agent 5 predictive alerts panel

File: `lib/agents/agent5-predictive.ts`

Admin-triggered predictive analysis powered by **Gemini 2.5 Pro**:

1. Fetches issues within a configurable lookback period (30/60/90 days).
2. Decodes PostGIS WKB location strings via `wkx`.
3. Runs **geographic DBSCAN clustering** (`lib/utils/clustering.ts`) on coordinates using **Haversine distance** — clusters are constrained to a maximum 50 km radius (one Indian district). The number of clusters is fully data-driven with no fixed upper limit; geographically distant cities always form separate clusters.
4. For each cluster, summarises historical issues (category, severity, dates) and prompts Gemini 2.5 Pro for forward-looking predictions.
5. Stores results in `predictive_alerts` (PostGIS POINT centroid, confidence 0–1, `basis_summary`).
6. Displayed as a heatmap overlay in the admin dashboard.

> **Key Insight**
>
> **Why geographic DBSCAN over K-Means:** The previous approach used K-Means with Euclidean lat/lng distance, which treats 1° of longitude the same as 1° of latitude — a significant distortion at India's latitudes. Additionally K-Means required a fixed cluster count (capped at 10), causing Mumbai and Pune (360 km apart) to merge into one cluster. The current algorithm uses Haversine distance in metres with a 50 km district-level cap, producing geographically accurate, data-driven clusters without any fixed limit.

4.9 LangGraph StateGraph Wiring

File: `lib/agents/pipeline.ts`

The pipeline uses LangGraph's `StateGraph` with 16 typed channels in `AgentState`: `reportId`, `rawText`, `originalLanguage`, `englishTranslation`, `translationTrace`, `imageUrl`, `videoUrl`, `coordinates`, `userId`, `classification`, `deduplication`, `validation`, `resolution`, `error`, `imageAnalysis`, `address`.

Background execution: The pipeline is invoked via Next.js 16's `after()` — it runs asynchronously after the HTTP response is returned. The `/api/reports/process` endpoint returns immediately with the `issueId` while agents process in the background.

Pipeline retry/resume: If any agent fails (LLM timeout, network error), the issue's `pipeline_stage` is set to `*_failed`. The `/api/reports/retry` endpoint reconstructs the full `AgentState` from database columns and resumes from the failed agent — not from the beginning.

5 Google Technologies Used

5.1 Gemini 2.5 Flash — Agents 0 through 4

Every agent uses `generateStructuredJSON<T>()` with `temperature: 0.1` for deterministic output. Each agent has a carefully crafted system prompt constraining the model to valid JSON only, no markdown, no preamble.

5.2 Gemini 2.5 Pro — Agent 5

Reserved exclusively for predictive analysis. The `usePro` flag in the unified AI client switches from `gemini-2.5-flash` to `gemini-2.5-pro`. This is reserved for complex multi-step reasoning where accuracy matters more than latency.

5.3 Gemini Embedding (gemini-embedding-001)

Generates 768-dimensional embeddings in two places: (1) on report submission for storage in the `issues.embedding` pgvector column, and (2) at Agent 2 runtime for vector similarity comparison. `outputDimensionality: 768` is set explicitly to match the pgvector column.

5.4 Gemini Flash Vision — Agent 1

Photos uploaded with reports are analysed via Gemini's multimodal API. Images are first optimised by `sharp` (1024px max, JPEG 85%) then sent as base64 inline data. This provides visual evidence for classification: a photo of a broken streetlight yields "Damaged pole with exposed wiring — safety hazard on residential road," which is combined with the text description for more accurate severity scoring.

5.5 Google Maps Geocoding API

File: `app/api/geocode/route.ts`. Reverse-geocodes GPS coordinates into human-readable addresses by parsing `address_components` from the Maps API for `locality` (city) and `administrative_area_level_1` (state). Returns a clean "City, State" string in real time as the citizen selects a location.

5.6 Google Maps Routes API — TSP Optimisation

File: `app/api/admin/routes/optimize/route.ts`. Accepts an origin point and issue waypoints and calls `computeRoutes` with `optimizeWaypointOrder: true`, solving the Travelling Salesman Problem for minimal total travel distance. Returns the optimised order, total duration/distance, and an encoded polyline for Mapbox rendering.

5.7 Google Cloud Run — asia-south1

- **Region:** asia-south1 (Mumbai) for low latency to Indian users
- **Live URL:** `https://civicpulse-948264080742.asia-south1.run.app`
- **Configuration:** `--allow-unauthenticated`; environment variables via `cloudrun-env.yaml`; port 3000; non-root container user

5.8 Google Artifact Registry

Docker images tagged with Git SHA pushed to `asia-south1-docker.pkg.dev/civicpulse-500507/civicpulse-repo/civicpulse`.

5.9 Workload Identity Federation

The CI/CD pipeline authenticates to GCP with **zero stored credentials** — GitHub's OIDC token is exchanged for a short-lived GCP access token. Pool: `projects/948264080742/locations/global/workloadIdentityPools/github-pool`. No service account JSON keys stored anywhere.

5.10 Google OAuth

Supabase Auth is configured with the Google OAuth provider (`NEXT_PUBLIC_GOOGLE_CLIENT_ID`). The callback at `app/auth/callback/route.ts` exchanges the code for a session and reads a `role` query parameter to route admins to `/admin/dashboard`.

5.11 Google Antigravity & MCP Tooling

During development, the platform was built and accelerated using Google's advanced AI coding ecosystem, specifically the **Antigravity IDE** and **Antigravity Agents** (including Agent Ali). To enable autonomous browser testing and context-aware development, the **Google Stitch MCP Server** and the **Chrome DevTools MCP Server** were heavily leveraged, allowing the AI to seamlessly interact with local development servers and debug web interfaces in real time.

6 Innovation and Creativity

6.1 Dual-Provider AI Architecture

A single environment variable (`AI_PROVIDER=gemini` or `ollama`) switches the entire AI backend. The unified client (`lib/ai/client.ts`) abstracts over both providers:

Feature	Gemini (Default)	Ollama (Alternate)
Flash model	`gemini-2.5-flash`	`qwen3:30b-a3b`
Pro model	`gemini-2.5-pro`	`qwen3:235b-a22b`
Vision model	`gemini-2.5-flash`	`llava`
Embedding model	`gemini-embedding-001`	`nomic-embed-text`

This enables development without burning API credits and demonstrates vendor-agnostic architecture.

6.2 Cloudflared Tunnel for Hybrid AI

File: `scripts/manage-gcloud.ps1`

Solves the problem of routing Cloud Run traffic through a local GPU running Ollama:

1. Starts `cloudflared tunnel --url http://localhost:11434` in background.
2. Parses the assigned temporary HTTPS URL from the log.
3. Updates `.env.production` and pushes to GitHub Secrets via `gh secret set`.
4. Triggers a new Cloud Run deployment automatically.

One command connects a developer's local GPU to a globally deployed cloud application.

6.3 Three-Layer Deduplication

No existing civic platform combines vector similarity, spatial proximity, and semantic AI verification. Each layer catches cases the others miss — see §4.5 for details.

6.4 Universal Language Support (Agent 0)

India's linguistic diversity is a major barrier for civic engagement. CivicPulse breaks this barrier by running all inputs through **Agent 0**, which automatically detects the input language and translates it into English before it enters the main LangGraph pipeline. It supports *any* language supported by Gemini. When the pipeline resolves, the final civic brief and status updates are translated back into the citizen's original language in the dashboard.

6.5 Predictive Hotspot Clustering (DBSCAN)

To identify emerging civic hotspots before they escalate, CivicPulse uses a custom geographic DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm. This is calculated directly in `lib/utils/clustering.ts` using the Haversine formula to account for the Earth's curvature.

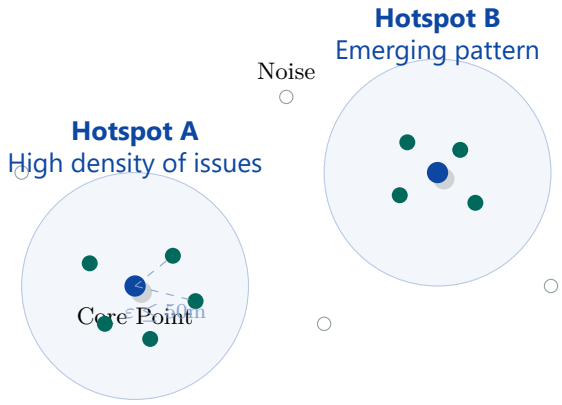

Figure 7: DBSCAN algorithm categorises closely grouped civic issues into actionable hotspots, while filtering out isolated noise incidents.

The algorithm identifies dense regions of issues (Core Points) within an ε-radius, groups them, and passes the aggregate data to Agent 5 (Gemini 2.5 Pro) to generate predictive maintenance alerts.

6.6 Karma Gamification

Event	Points	Rationale
Report an issue	+10	Incentivise reporting
Participate in community review	+2	Incentivise verification
Vote with the majority	+5	Reward accurate judgment
Report confirmed by community	+20	Reward legitimate reports
Issue resolved by municipality	+50	Celebrate civic impact
Report rejected by community	–5	Penalise spam/false reports

6.7 Impact Receipt

When an issue is resolved, citizens generate a shareable OG image (1200×630px) via `next/og` showing: issue title, location, fix time, estimated citizens affected, and CivicPulse branding — designed for WhatsApp and social media sharing.

6.8 Community Verification with Quorum

Issues with credibility < 6 enter `community_review`. Nearby citizens vote verify/deny. Quorum rules: minimum 3 votes; if confirms $\geq$ denies the issue resumes the pipeline at Agent 4. If denies $>$ confirms the issue is closed and the reporter loses 5 karma. The vote endpoint internally calls `/api/reports/retry` on confirmation.

6.9 Route Optimisation with TSP

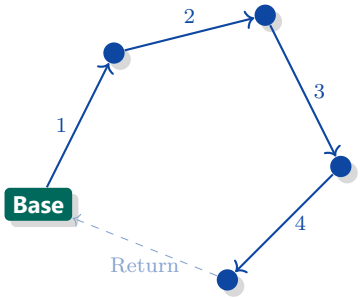

Figure 8: TSP Route Optimisation computes the most efficient path for municipal workers to address clustered issues in a single trip.

The admin Route Planner clusters open issues geographically and computes an optimal visit sequence for municipal field teams via Google Maps Routes API `optimizeWaypointOrder: true`. The encoded polyline is rendered on a Mapbox map.

6.10 Interactive Geospatial Visualisation

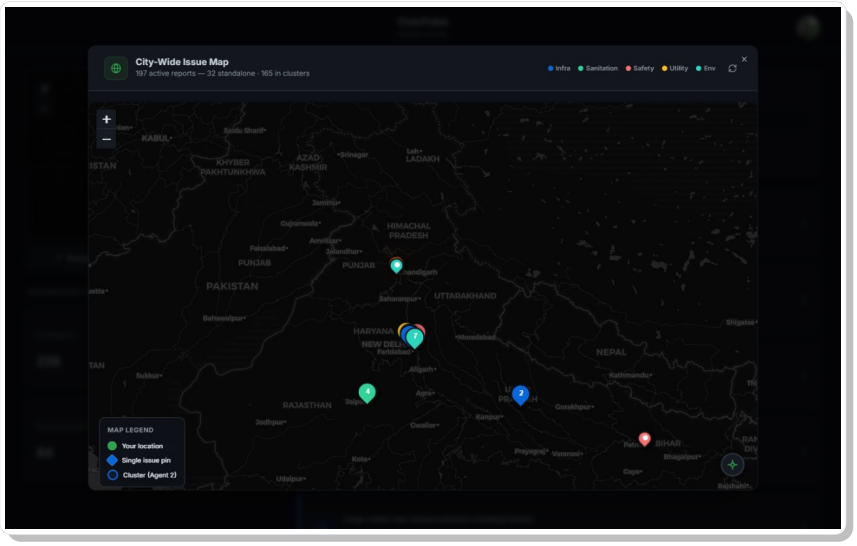

Figure 9: Interactive Geospatial Map — Real-time issue clustering for citizens and administrators

CivicPulse leverages Mapbox GL JS and GeoJSON to render an interactive, real-time map of all civic issues across the region. Unlike traditional platforms that restrict geospatial data to administrators, this interactive clustering map is exposed to **both citizens and administrators**.

By providing citizens with a bird's-eye view of civic health in their locality, we foster community awareness and transparency. The map dynamically clusters nearby issues to maintain performance and prevent visual clutter, elegantly unpacking into individual issue pins as the user zooms in.

7 Product Experience and Design

7.1 Design Philosophy

CivicPulse follows a **GitHub-inspired design system**: high information density, functional colour usage, and zero decorative elements. The philosophy: a civic platform used daily by municipal workers must prioritise clarity over aesthetics.

- **Dark mode by default:** `#0d1117` background, `#c9d1d9` text, `#161b22` cards, `#30363d` borders — exactly GitHub's dark palette.
- **Functional colours:** Green (`#2da44e`) for success; Blue (`#0969da`) for primary actions; Red (`#cf222e`) for emergencies. No decorative gradients.
- **Inter font** via `next/font/google` — clean, legible, widely supported.
- **shadcn/ui (Radix primitives)** for keyboard navigation and ARIA accessibility.

7.2 Mobile-First Responsive Design

CivicPulse is built with a **mobile-first approach** — every screen is designed for a 375px handset viewport first and enhanced for wider screens. This is deliberate: the typical Indian citizen reporting a pothole is holding a smartphone, not sitting at a desktop.

- **Fluid layouts:** Tailwind's `sm:`, `md:`, `lg:` breakpoints provide seamless adaptation from 320px feature phones to 1920px admin monitors.
- **Touch targets:** All interactive elements meet the WCAG 44×44px minimum tap target size.
- **Report page on mobile:** Text area fills the screen; GPS, camera, and image upload are one-tap actions designed for outdoor use without fine motor precision.
- **Dashboard on mobile:** Issue cards collapse by default, expanding on tap; pipeline stepper is horizontally scrollable; mini-map opens as a full-screen overlay.
- **Admin dashboard on mobile:** The full admin inbox, map, Agent 5 panel, and Route Planner all reflow to single-column layouts on small screens — tested across multiple viewport widths.
- **Dark OLED mode:** The `#0d1117` background reduces power consumption significantly on AMOLED/OLED phone displays, extending battery life during field use.
- **PWA-ready:** A service worker stub (`public/sw.js`) and web manifest are in place for future offline support and home-screen installation.

7.3 Loading Animations & Pipeline Visibility

To ensure citizens do not feel abandoned while the complex six-agent pipeline runs in the background, CivicPulse implements dynamic loading states. While images are being processed by Gemini Vision (Agent 1) and while the vector database checks for duplicates (Agent 2), users are presented with smooth, reassuring loading animations.

This transparent pipeline visibility actively communicates that "work is being done," building trust and patience. The interface visually transitions through each stage—Classification, Deduplication, and Validation—providing real-time feedback as the AI completes its reasoning steps.

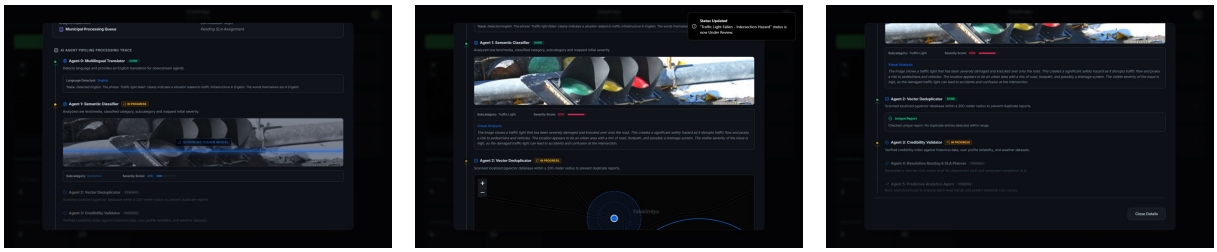

Figure 10: Dynamic pipeline UI showing progress through Deduplication, Validation, and Resolution.

8 User Perspective — Accessibility and Inclusivity

8.1 Non-English Speaker User Story

Consider **Sunita**, a homemaker in Nashik who writes only in Marathi:

1. **Login:** Opens the app, taps "Citizen," signs in with Google (consent screen renders in her device language). No English required.
2. **Reporting:** She sees a burst water pipe. She opens the Report page and *types* in Marathi: "**माझ्या गल्लीत पाण्याची पाइप फुटली आहे, पाणी वाहत आहे.**" She taps the webcam button to attach a photo and hits "Use GPS."
3. **Agent 0:** Detects Marathi, translates to English: "A water pipe has burst in my lane, water is flowing." Translation trace stored.
4. **Agents 1–4:** Classify as utility/water_leakage, severity 7, department: Water Supply. Deduplication passes. Credibility 9/10 (photo + GPS). Civic brief generated in both English and Marathi.
5. **Dashboard:** Sunita's `local_civic_brief` is displayed in Marathi — she reads the municipality's action plan in her own language.
6. **Resolution:** Pipe fixed → +50 karma → shareable Impact Receipt for WhatsApp.

8.2 Accessibility Features

Feature	Benefit
Type in any language	Agent 0 automatically translates; no English required
Voice input (English)	Citizens can speak in English; text is transcribed by the browser's Web Speech API
Localised civic brief	Resolution summaries in the citizen's own language
GPS auto-detection	No need to type an address
Webcam capture	No separate camera app needed
Drag-and-drop upload	Intuitive on all devices
Real-time pipeline tracking	Citizens know exactly what is happening
Mobile-first layout	Fully usable on any smartphone
Dark OLED mode	Reduces battery drain on AMOLED screens
Keyboard navigation (Radix)	Full keyboard + screen reader support

8.3 Web Accessibility (a11y) & Screen Reader Support

CivicPulse heavily leverages HTML ARIA attributes and semantic structure to ensure the platform is fully usable for visually impaired citizens and those relying on keyboard navigation:

- **Screen Reader Labels:** All icon-only interactive elements (like the "Use GPS location" and "Choose location on map" buttons) are explicitly tagged with `aria-label`, ensuring screen

readers articulate the exact action rather than skipping the icon.

- **Tab Navigation & Roles:** Complex UI components like dashboard status/category filters use strict WAI-ARIA patterns including `role="tablist"`, `role="group"`, `aria-selected`, and `aria-pressed`. This allows users to tab through filters and intuitively understand the active state entirely via keyboard navigation.
- **Dynamic Announcements:** For asynchronous background tasks (like the AI pipeline processing), visually hidden `sr-only` containers with `aria-live="polite"` and `aria-atomic="true"` are used. This allows the UI to silently announce status changes (e.g., "Issue deduplication complete") to screen readers without jarring visual interruptions.

9 Technical Implementation

9.1 Unified AI Client

File: `lib/ai/client.ts` (318 lines)

Single point of contact for all AI operations with four exported functions:

Function	Purpose	Used By
`generateStructuredJSON<T>(...)`	Typed JSON from LLM	All 6 agents
`analyzeImage(url, prompt)`	Multimodal vision	Agent 1
`generateEmbedding(text)`	768-dim embedding	Submission, Agent 2
`generateText(prompt, system?)`	Plain text	Utility

Key implementation details: Singleton pattern for client instances; custom `fetch` with `keepalive: false` to prevent `UND_ERR_SOCKET` behind tunnels; JSON parsing strips markdown fences before `JSON.parse()`.

9.2 Agentic Retry Logic & Self-Correction

To ensure the pipeline is highly resilient to LLM hallucinations and malformed outputs, the architecture implements a multi-tiered agentic retry mechanism:

- **Syntax Level (Self-Correction):** If an agent outputs invalid JSON or hallucinates extra text, the `generateStructuredJSON` function catches the `SyntaxError`, feeds the exact error message back into a new prompt, and asks the LLM to self-correct its output (up to 3 attempts).
- **API Level (Exponential Backoff):** Temporary API rate limits or network timeouts are handled via exponential backoff retries within the AI client.
- **State Level (LangGraph Resume):** If all in-node agentic retries fail, the LangGraph pipeline safely halts and persists the state as `*_failed` in the database. When triggered via the `/api/reports/retry` endpoint, the pipeline reconstructs the full `AgentState` and resumes precisely from the failed agent, ensuring that expensive previous steps (like image analysis or embeddings) are never re-run unnecessarily.

9.3 Three Supabase Clients

Client	Auth Level	Usage
`createClient()`	Anon key (RLS)	Browser components
`createServerSupabaseClient()`	Anon + cookie session	Server components, API routes
`createServiceClient()`	Service role (RLS bypassed)	Agent pipeline (no user session)

9.4 Auth Flow

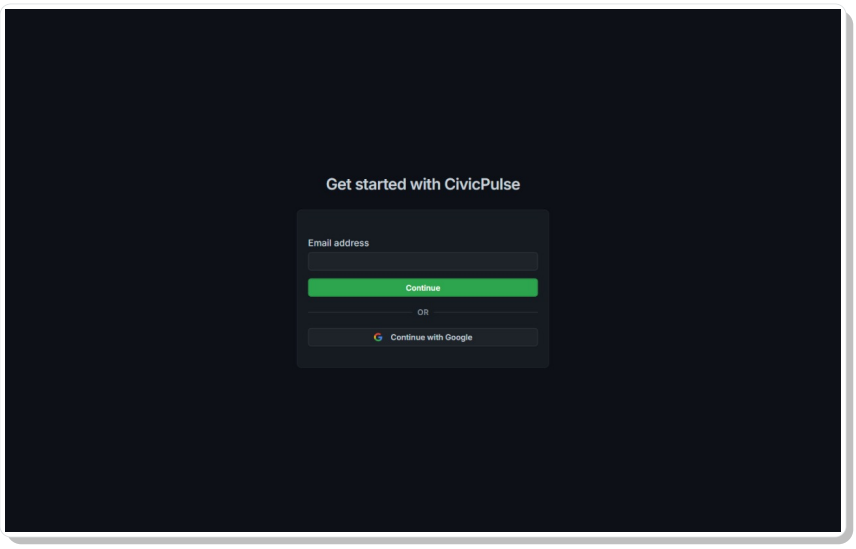

Figure 11: Login — Google OAuth + email magic link

The OAuth flow: Google consent → callback `/auth/callback` → `exchangeCodeForSession()` → role-based redirect. The `role=admin` query parameter in the redirect URL sets the user's profile role and sends them to `/admin/dashboard`.

Auth middleware (`proxy.ts`) protects `/dashboard`, `/report`, and `/admin`. For `/admin` paths, it additionally queries the `profiles` table to verify `role='admin'`.

9.5 Community Verification System

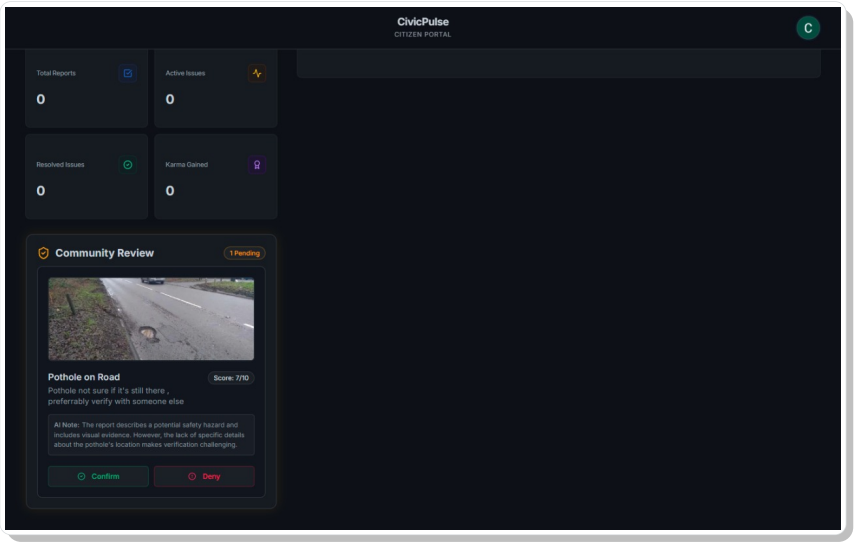

Figure 12: Community verification — verify/deny panel

Files: `app/api/reviews/route.ts`, `app/api/reviews/[issueId]/vote/route.ts`

1. **Fetch reviewable issues:** PostGIS `ST_DWithin` within 1 km of the user's saved home location; excludes their own reports and already-voted issues.
2. **Cast vote:** Records verdict, awards +2 karma, checks quorum (min 3 votes).

3. **Quorum resolution:** Confirm majority → auto-resume pipeline at Agent 4 (`/api/reports/retry`). Deny majority → close issue, –5 karma to reporter.

9.6 PostGIS WKB Decoding

Supabase returns geography columns as hex-encoded WKB. Four routes decode via `wkx`: `wkx.Geometry.parse(Buffer.from(hex, 'hex')).toGeoJSON()`, extracting `coordinates[1]` (lat) and `coordinates[0]` (lng). WKT text format handled as fallback.

10 Database Schema

10.1 Entity Relationship Diagram

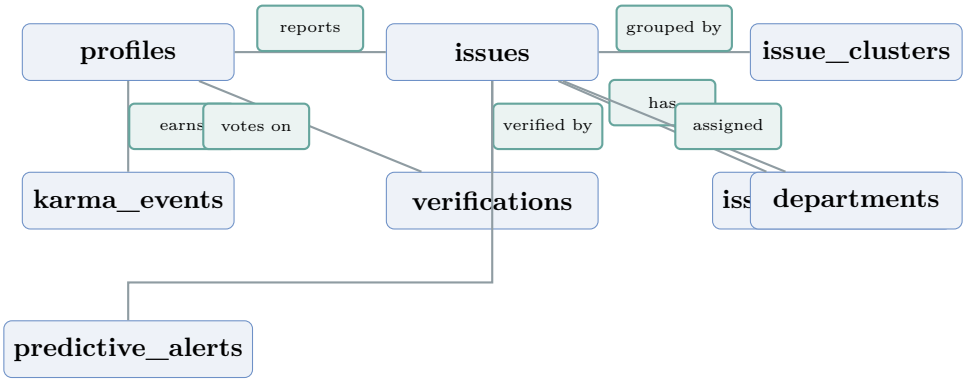

Figure 13: Database entity relationships — 7 tables with PostGIS geography and pgvector extensions

10.2 Key SQL RPC Functions

Function	Purpose
`match_issues(embedding, threshold, count)`	pgvector cosine similarity search — returns top N issues above threshold
`issues_within_radius(lat, lng, radius, category)`	PostGIS `ST_DWithin` — spatial proximity search
`increment_cluster_count(cluster_id)`	Atomic cluster size counter

11 DevOps, CI/CD, and Cloud Deployment

Key Insight

Live deployment: `https://civicpulse-948264080742.asia-south1.run.app`

11.1 CI/CD Pipeline

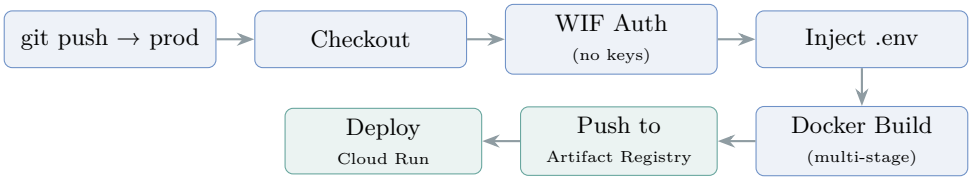

Figure 14: GitHub Actions CI/CD: push to `prod` → WIF auth → Docker build → Artifact Registry → Cloud Run

Trigger: Push to the `prod` branch.
Auth: `google-github-actions/auth@v2` with Workload Identity Federation — no service account JSON anywhere.
Docker build: Passes all `NEXT_PUBLIC_*` vars as build args (Next.js inlines these at build time).
Deploy: `google-github-actions/deploy-cloudrun@v2` with `--allow-unauthenticated`.

11.2 Multi-Stage Dockerfile

Stage	Base Image	Purpose
`deps`	`node:20-alpine`	Install Bun, run `bun install --frozen-lockfile`
`builder`	`node:20-alpine`	Copy source, set `NEXT_PUBLIC_*` build args, run `bun run build`
`runner`	`node:20-alpine`	Non-root user (`nextjs:1001`), copy standalone output, pre-install `sharp`

12 Completeness and Usability

12.1 End-to-End Citizen Flow

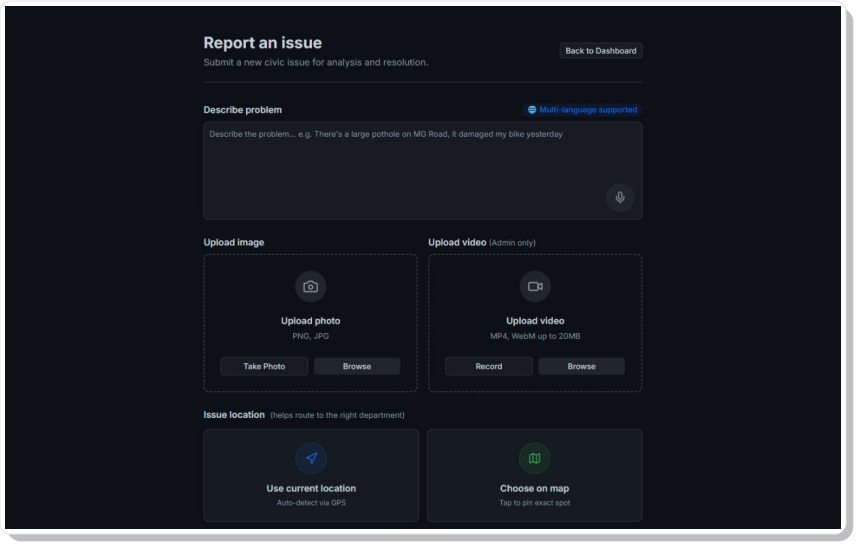

Figure 15: Report page — text input + voice button

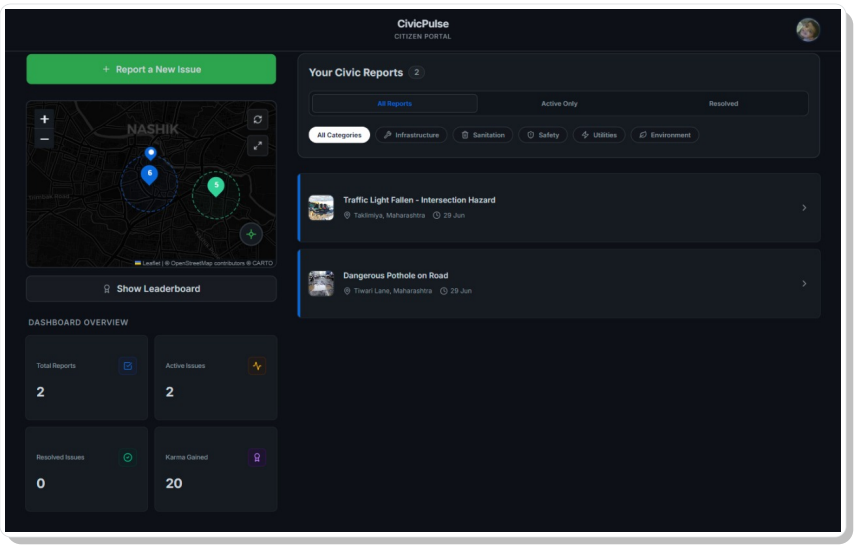

Figure 16: Citizen dashboard — issue cards with pipeline stepper

#	Feature	Implementation
1	Authenticate	Google OAuth or email magic link via Supabase Auth
2	Report issue	Text (any language), photo/video (file, drag-drop, webcam), voice (English), GPS or map picker
3	Track processing	Real-time pipeline stage updates via Supabase Realtime
4	View results	Category, severity, civic brief (English + local language), SLA deadline, department
5	Community verify	View nearby low-credibility issues; cast verify/deny votes with comments
6	Earn karma	Points for reporting, reviewing, getting confirmed, issue resolved
7	Leaderboard	Top 10 citizens by karma; personal rank displayed
8	Impact Receipt	Shareable OG image for social media
9	Retry failed	Pipeline auto-resumes from last successful agent

12.2 End-to-End Admin Flow

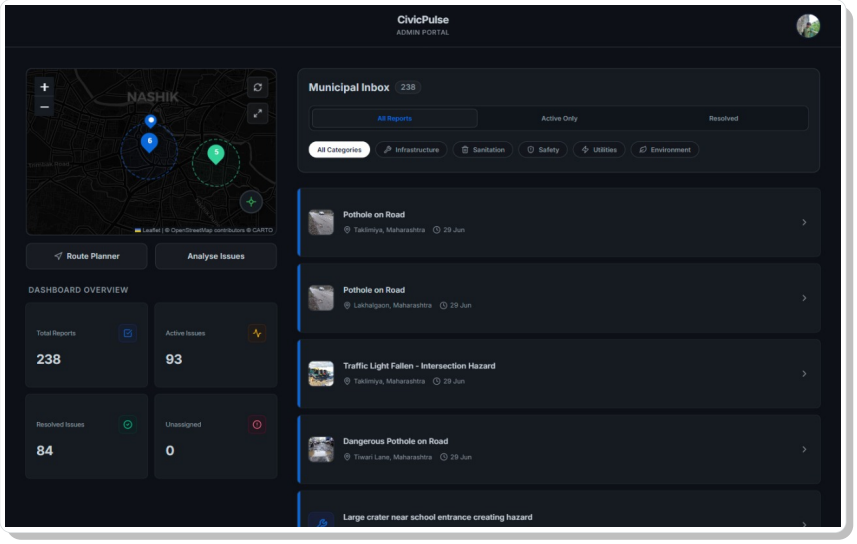

Figure 17: Admin dashboard — issue inbox with filters

#	Feature	Implementation
1	Admin login	Google OAuth with `role=admin` parameter
2	Issue inbox	Filterable issue list with real-time updates
3	Manage issues	Status updates, department assignment, comment thread
4	Map view	Mapbox issue pins with geographic DBSCAN cluster circles
5	Predictive analysis	Agent 5 with configurable lookback; heatmap overlay
6	Route optimisation	TSP routing for field teams via Google Maps Routes API

12.3 Test Coverage

Twelve dedicated test files in `tests/`: agents 1–4, DB connectivity, geolocation, Gemini vision, Ollama vision, update operations, community voting, and pipeline retry.

13 What Makes CivicPulse Different

Feature	Municipal Portals	311 Apps (US)	CivicPulse
AI classification	–	Keyword matching	**6-agent LangGraph + Gemini**
Duplicate detection	–	Text matching only	**Vector + spatial + AI**
Multilingual (typed)	Separate portals	English only	**Auto-detect + translate**
Predictive analytics	–	–	**Geographic DBSCAN + Gemini Pro**
Community verification	–	–	**Quorum-based + karma**
Real-time tracking	Email only	Push notifications	**Supabase Realtime**
Route optimisation	–	–	**Google Maps TSP**
Mobile-first design	Rarely	Some	**Full mobile-first + OLED**

14 Future Roadmap

Priority	Feature	Description
High	WhatsApp Integration	Citizens report via WhatsApp (text+photo+location) → same pipeline via webhook
High	Web Push Notifications	Infrastructure already in place (VAPID keys, push_subscription column)
Medium	Scheduled Agent 5	Cloud Run scheduled job (cron) instead of manual admin trigger
Medium	Ward-Level Analytics	Resolution rates, SLA compliance, issue distribution by municipal ward
Low	False Closure Detection	Citizens challenge “resolved” status if issue persists
Low	Multi-City Support	Configuration-driven city selection with per-city department databases

15 File Reference

15.1 Application Pages

File	Lines	Purpose
app/layout.tsx	26	Root layout: Inter, dark mode, Toaster
app/page.tsx	32	Landing: role selection, auth redirect
app/auth/login/page.tsx	126	Login UI: Google + email magic link
app/auth/callback/route.ts	28	OAuth code exchange, role-based redirect
app/report/page.tsx	563	Report: text, voice, image, video, GPS, map
app/dashboard/page.tsx	39	Citizen dashboard server component
app/admin/dashboard/page.tsx	40	Admin dashboard server component
app/receipt/[issue_id]/route.tsx	118	OG Impact Receipt (1200×630, edge runtime)

15.2 API Routes

File	Lines	Purpose
api/reports/process/route.ts	88	Pipeline entry: insert + embed + run
api/reports/retry/route.ts	109	Resume failed pipeline
api/issues/map/route.ts	52	All issues with decoded WKB locations
api/reviews/route.ts	162	Community-reviewable issues within 1 km
api/reviews/[issueId]/vote/route.ts	147	Cast vote, quorum, auto-resume
api/admin/clusters/route.ts	81	Geographic DBSCAN clusters for admin map
api/admin/predictive/run/route.ts	43	Trigger Agent 5
api/admin/routes/optimize/route.ts	89	Google Maps Routes TSP
api/leaderboard/route.ts	80	Top 10 citizens by karma

15.3 Agent and Library Files

File	Lines	Purpose
`lib/ai/client.ts`	318	Unified AI: Gemini + Ollama, 4 functions
`lib/agents/agent0-translation.ts`	79	Language detection + translation
`lib/agents/agent1-classifier.ts`	100	Multimodal classification
`lib/agents/agent2-deduplication.ts`	178	Three-layer deduplication
`lib/agents/agent3-validation.ts`	161	Weather-aware credibility scoring
`lib/agents/agent4-resolution.ts`	140	Civic brief + SLA computation
`lib/agents/agent5-predictive.ts`	158	Geographic DBSCAN + Gemini Pro
`lib/agents/pipeline.ts`	124	LangGraph StateGraph wiring
`lib/utils/clustering.ts`	174	Geographic DBSCAN (Haversine, 50 km cap)
`proxy.ts`	57	Auth middleware: route + role protection

CivicPulse

Problem Statement 2: Community Hero — Hyperlocal Problem Solver

Submitted by **Ronit Sonawane** · B.Tech CSE, IIT Guwahati

github.com/ronits2407/civicpulse | June 2026